Typed Adaptive Context for LLM Agents: Reversible Records, Progressive Materialization, and Stateful Cursors

Jorge Simão
EInnovator R&D – Center for the Sciences of Computation, Cognition, and Complexity

August 2026

Abstract

An agent does not observe one homogeneous token stream. It receives tool responses, database rows, logs, terminal output, retrieved passages, graph neighborhoods, files, and paginated API results. Sending every result in full wastes prompt and native key/value capacity; replacing each result with one lossy summary can hide the low-salience fact that should trigger a later action. We introduce a typed adaptive-context runtime that separates four objects often conflated in context managers: a compact prompt-visible view, retrieval-only address views, exact scoped backing state, and a materialization policy. A fifth object, a stateful cursor, supports bounded analysis of large collections. Exact payloads are content-addressed, hash-verified, tenant/session scoped, TTL bounded, and recovered by identity rather than semantic rediscovery. Type-specific compressors preserve schemas, representative rows, numeric statistics, errors, command status, and graph structure while lexical, entity, rare-term, and schema indices remain available for discovery.

We report an inception mechanism study over 45 typed records and 5 seeds. Compact views save 0.633 of serialized bytes on average and retain only 0.267 of planted low-salience triggers. Exact backing recovery and explicit address recovery are both 1.000, but a generic latent query recovers 0.000. A proactive probe that already names the candidate action reaches 1.000, locating the remaining difficulty in deciding *what to probe*, not in storage reversibility. On a 100-row result, full native-event and tool materialization each transfer 3776 bytes, whereas a five-row mixed cursor transfers 180 bytes. Cursor aggregate, filter, and anomaly checks are exact on five fixtures. These are deterministic mechanism results, not model task accuracy. They support typed reversible context as a substrate while leaving latent trigger discovery and end-to-end agent behavior as explicit open gates.

1 Introduction

Tool-capable language-model agents increasingly spend their context budget on observations rather than instructions. A database query can return thousands of rows; a build command can produce megabytes of mostly routine output; a graph traversal can expose a neighborhood whose useful branch is not yet known. The agent often needs only a small projection now, but it may need a different projection after its next inference.

Two common responses are unsatisfactory. Full serialization couples active context to source size. One-shot summarization breaks that coupling, but the summary becomes both the model's visible evidence and its only clue that more evidence exists. An exact original can remain in a cache while being functionally unreachable: the model cannot request a detail whose relevance was itself omitted.

This paper treats context management as a data-system problem with a model in the control loop. A result is a typed record, not an undifferentiated message. Its initial representation, search addresses, exact state, and transfer policy are separate. Large collections may expose cursors with bounded operations instead of repeatedly returning complete snapshots. The architecture joins the

typed capability records developed for tool and skill disclosure with typed *result* records that survive after execution.

The contributions of this inception release are:

1. a provider-neutral record model for nine result families, alongside the inherited tool and skill capability records;
2. a scoped content-addressed backing store with exact recovery, hash verification, TTL, size bounds, persistence controls, and audit events;
3. deterministic type-specific compactors and independent lexical, entity, rare-term, and schema address views;
4. metadata, compact, selected, and full materialization levels exposed through native-event and tool-compatible APIs;
5. stateful cursors supporting pages, ranges, filters, ordering, search, aggregation, sampling, field projection, and closure;
6. a five-seed mechanism benchmark that measures savings, addressability, exact recovery, cursor correctness, and transport accounting while keeping model-dependent claims out of the result.

The main negative result matters as much as the implementation. Multiple address views make a named hidden detail recoverable. They do not tell a model which unnamed detail should alter its next action. Paper 7 therefore separates *recovery capability* from *recovery policy*.

2 Context Is a Typed State Space

Let a result record be

$$R = (id, \tau, P, A, V, B), \tag{1}$$

where id is a stable scoped identity, τ is a record type, P contains provenance and policy, A is a set of address views, V is the current prompt-visible projection, and B is lossless backing state. The model need not see A or B directly. They belong to the runtime's control plane.

This separation prevents three category errors. First, a compact view is not a retrieval index: prose that reads well may be a poor rare-key index. Second, a cache handle is not authorization: identity resolution must still enforce tenant and session scope. Third, storage is not materialization: retaining bytes says nothing about when they enter the model's active context.

Materialization follows a monotone visibility lattice:

$$V_0 = \text{metadata} \to V_1 = \text{compact} \to V_2 = \text{selected fields/range} \to V_3 = \text{full}. \tag{2}$$

The backing record does not change as a view advances. A selected row range is computed from the hash-verified original after authorization, so an earlier summary cannot silently become the source of truth.

3 Typed Result Records

Table 1 lists the initial result families. The registry is open: applications may replace a compressor without changing identity, storage, authorization, or cursor semantics.

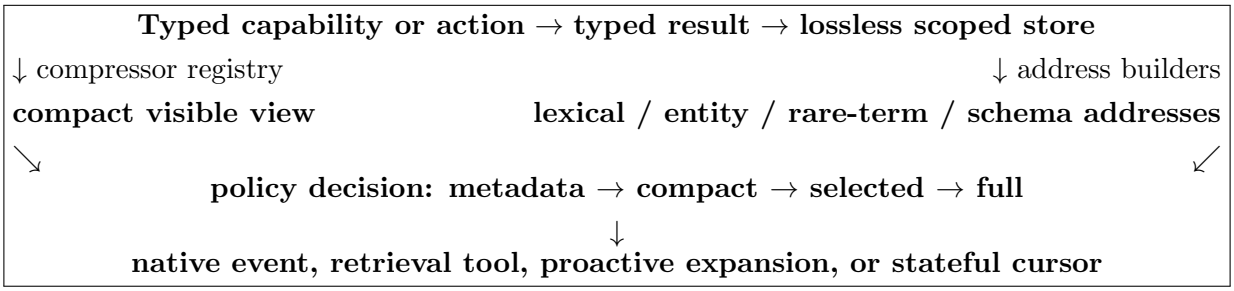

Figure 1: Paper 7 keeps prompt visibility, retrieval addresses, and exact backing state separate. Arrows into materialization are policy decisions, not claims that the model will always make the correct request.

Type	Compact view	Useful address/selection units
Tool/API response	bounded structural fields	keys, values, entities, rare identifiers
Log block	errors plus deterministic lines and counts	line ranges, severity, rare tokens
Terminal output	command, status, stderr, stdout head/tail	stdout/stderr ranges, command identity
Database result	schema, representative rows, numeric statistics	fields, row ranges, filters, aggregates
Graph result	node/edge counts and representatives	node IDs, entities, neighborhoods, edge ranges
RAG result/chunks	deterministic passages	document/chunk identity, lexical and entity views
File read	deterministic line projection	path, line ranges, rare keys
Generic text	deterministic line projection	lexical, entity, and rare-term views
Tool/skill capability	inherited selection and full views	stable capability URI

Table 1: Initial type policies. Every lossy row retains an exact original.

3.1 Compact views

The inception implementation uses deterministic compressors because their failure modes are inspectable. Database results retain column names, row count, uniformly spaced representative rows, and min/max/mean statistics for numeric columns. Logs prioritize lines matching error, exception, failure, fatal, panic, denial, and timeout patterns before filling the remaining budget with deterministic samples. Terminal records preserve command, exit status, working directory, stderr, and bounded stdout head/tail. Graph records retain counts and representative nodes and edges. Text-like records sample lines at fixed relative positions.

These policies are baselines, not universal definitions of importance. Error preservation helps logs in the current benchmark, while uniform text sampling sometimes lands on a planted RAG trigger and usually misses it. Optional generated summaries can be registered, but no generated summary is used in the reported study.

3.2 Address views

Address views are built from exact input at ingestion time and stay outside the prompt-visible compact payload. The current implementation emits deterministic lexical tokens, entity-like spans, within-record rare terms, and structural keys. The API reserves dense and native-Q/K views without pretending they are implemented in this checkpoint.

For query q and address view A_j, discovery is

$$\text{score}(q, R) = \sum_j w_j \, s_j(q, A_j(R)). \tag{3}$$

The inception lexical resolver uses exact normalized token overlap. It is deliberately simple: its role is to test whether omitted bytes remain addressable, not to establish a production semantic retriever.

4 Lossless Backing State

The local store serializes JSON-compatible values canonically and preserves text and byte payloads exactly. A SHA-256 content hash defines the backing identity within a tenant/session scope. Payload and manifest files reside under a hash of that scope. Every read checks the caller scope, TTL, file presence, and content hash before decoding.

The distinction between identity and authorization is operational. Knowing an opaque content-addressed record handle does not permit a different session to read it. Cursor reads repeat the same check. Deletion revokes both payload and manifest; expiration removes them lazily on access or during cleanup. A bounded store evicts oldest entries before accepting further state, and rejects one payload larger than its configured capacity.

The current backend is local filesystem storage. The interface and transport accounting allow a remote or distributed backend later, but this paper does not report a remote storage service, network fault injection, encryption at rest, or multi-process concurrency.

5 Materialization and Transport

Four retrieval modes share one authorization path.

Native typed event. The runtime accepts

$$\texttt{MATERIALIZE}(id,\ level,\ selector,\ cursor). \tag{4}$$

The identity is already known, so expansion does not rerun semantic search.

Tool retrieval. `retrieve_record` wraps the same event API for providers that expose ordinary function calling. Equal payload and authorization behavior makes the native event an optimization/interface choice rather than a semantic requirement.

Mixed response and cursor. The initial result supplies bounded data, schema, `cursor_id`, `has_more`, and provenance. Later requests fetch only pages, ranges, fields, aggregates, or matching rows.

Proactive expansion. The runtime may materialize a selected projection before the model asks. This can repair a known risk pattern, but it shifts the burden to a policy that must control false positives and transfer cost.

The storage policy is independent of retrieval mode. `upfront` places the original at the model/server side; `on_demand` retains it agent side until expansion; `adaptive` chooses using topology, byte size, and expected reuse. The study evaluates 100,000-byte and 1,000,000-byte thresholds as configuration values. They are not proposed constants.

6 Stateful Cursors

A cursor is a scoped capability over a collection, not a copy of that collection. Its state includes record identity, source, query, schema, collection name, position, page size, filters, ordering, total

estimate, creation and expiration times, provenance, authorization fingerprint, and a continuation handle. Supported operations are next/previous page, bounded range, search, filter, numeric aggregate, deterministic sample, selected-field projection, and close.

Cursors matter when the next useful projection depends on the previous result. An analytics agent can request an aggregate, notice an outlying subgroup, and then inspect only rows in that subgroup. A graph agent can inspect one neighborhood before continuing traversal. Repeating a full tool call would discard position and may recompute or mutate source state.

The current cursor manager is in-process and read-only. It does not yet provide transaction isolation, snapshot versioning, distributed leases, or recovery after process loss. Those are serving requirements, not properties inferred from the mechanism tests below.

7 The Trigger-Reachability Problem

Storage reversibility answers: "Can the original be recovered if its identity and relevance are known?" Agent behavior asks a harder question: "Will the model realize that an omitted detail should change what happens next?" We separate three cases:

1. **explicit insufficiency**: the query names the hidden concept;
2. **latent insufficiency**: the hidden concept is required but is absent from the query;
3. **action-trigger omission**: the hidden detail would select a different downstream action or tool.

For a compact view V_c, define

$$\text{TriggerRecall} = P(\text{required downstream action remains reachable} \mid V_c). \tag{5}$$

The definition intentionally includes more than answer overlap. A benchmark must observe whether evidence is visible, whether expansion occurs, whether the correct next action remains selectable, and whether an index or proactive policy repairs an omission.

8 Inception Experiment

8.1 Protocol

The study uses seeds $\{11, 23, 37, 53, 71\}$ and nine result types per seed, giving 45 records. Each payload contains one rare identifier and one required-action token placed away from ordinary high-salience content. Type policies receive a four-unit compact-view budget. No language model, learned compressor, embedding model, or native-Q/K router participates.

We record serialized bytes before and after compression, exact backing equality, trigger presence in the compact view, retrieval by an explicit rare identifier, retrieval by the generic latent query "continue safely," and retrieval by a proactive probe that names the candidate action. The last condition is an upper-bound mechanism check: it assumes a policy has already generated the correct candidate action. It is not an autonomous latent discovery result.

M4 compares four APIs on the same 100-row database shape. Native-event and tool conditions request full state. Mixed mode opens a five-row cursor page. Proactive mode selects rows 0–4. Reported timing is local Python wall time and is not a model/server latency claim. M5 filters a subgroup, computes its mean, and finds an anomaly through a cursor. M6 executes policy decisions for 10K, 100K, and 1M text payloads at low and high expected reuse across four declared topologies. The topology sweep tests accounting logic; it does not instantiate four physical deployments.

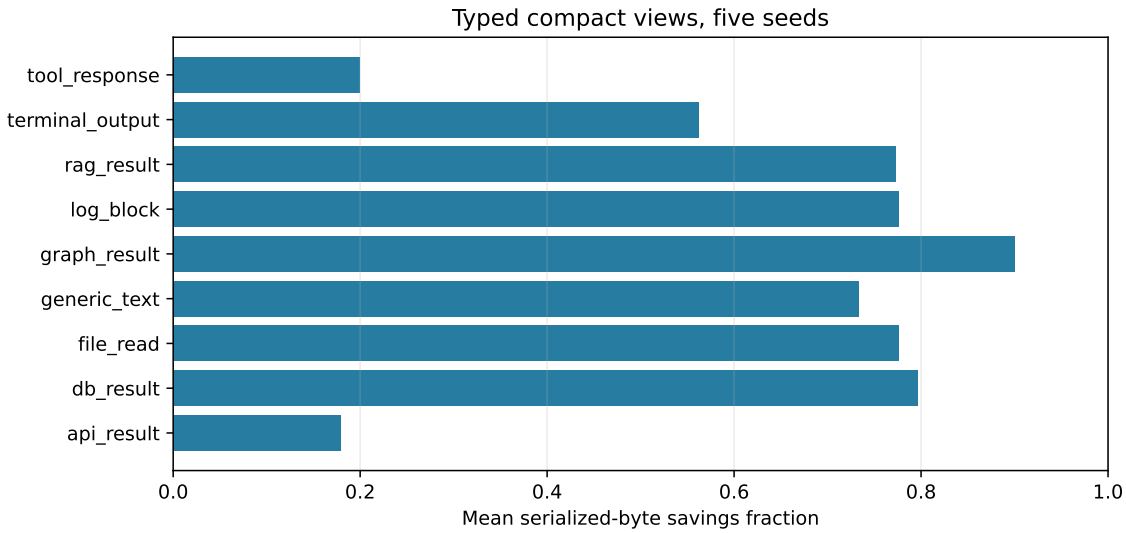

Figure 2: Mean serialized-byte savings by type across five seeds. Small structured tool/API fixtures have limited headroom; graph, database, log, and text-like results are substantially reduced.

Record type	Byte savings	Compact trigger recall
API result	.179	.000
Database result	.797	.000
File read	.776	.000
Generic text	.734	.000
Graph result	.900	.000
Log block	.776	1.000
RAG result	.773	.400
Terminal output	.562	1.000
Tool response	.200	.000

Table 2: Five-seed deterministic mechanism results. Trigger recall measures literal presence in the compact view, not semantic model use.

8.2 Compression

Across the nine equally weighted types, mean savings are 0.633. Table 2 reports type means. The graph fixture saves 0.900 and the database fixture 0.797; the small API and tool fixtures save only 0.179 and 0.200. This is expected: fixed metadata can exceed the removable content of a small result. The mechanism should therefore bypass compression below a measured break-even size in production.

8.3 Reversibility is not latent discovery

Figure 3 gives the central result. Compact views expose 0.267 of planted triggers. The error-preserving log and terminal policies retain all their triggers; deterministic RAG sampling lands on two of five; all other compactors miss them. The exact backing store recovers every original, and an explicit rare-token query finds every target through the address index.

The generic latent query finds none. The candidate-action probe finds all, but only because the correct action token is supplied to search. Thus the address layer repairs *omission after a usable discriminator exists*. It does not generate that discriminator. A safe default cannot be justified by

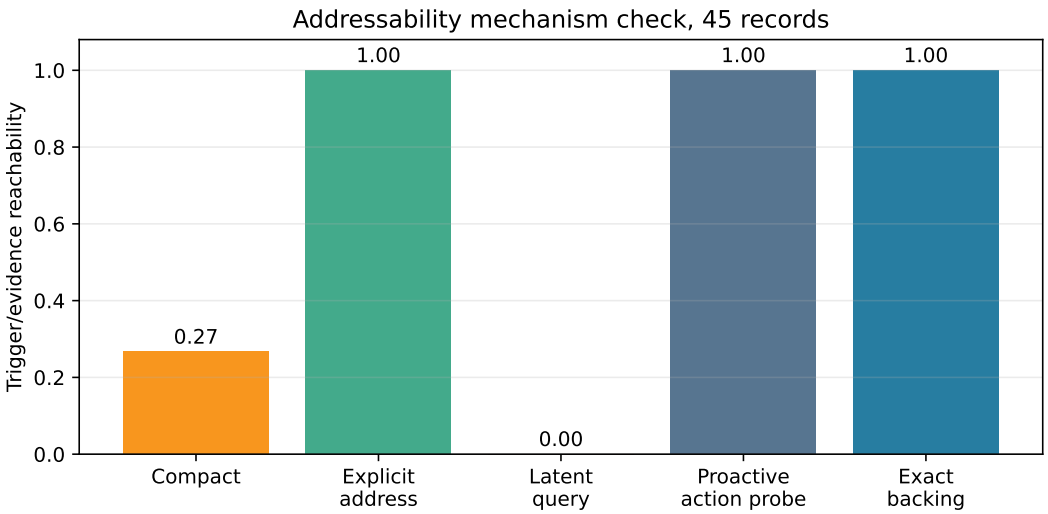

Figure 3: Compact visibility and addressability are different properties. Explicit and candidate-action probes use retrieval-only views; exact backing is hash-verified full recovery.

Mode	Mean transferred bytes	Row coverage	Median local time (ms)
Native full event	3,776	1.00	2.63
Tool full retrieval	3,776	1.00	2.44
Mixed five-row cursor	180	.05	3.21
Proactive five-row selection	244	.05	2.51

Table 3: Mechanism accounting on five seeded 100-row fixtures. These local times exclude model inference and real network transport.

reporting only exact recovery or explicit-query recall.

8.4 Retrieval modes and cursors

Native events and retrieval tools return identical full state and transfer 3776 bytes on average. This supports a narrow interface conclusion: a native event is not required for exact semantics when ordinary tool calling is available. The mixed cursor transfers 180 bytes for 0.050 row coverage, while selected proactive materialization transfers 244. Neither partial mode is "better" without a task: both save bytes by returning less data.

The five cursor analytics fixtures obtain 1.000 accuracy for filtered counts and means and 1.000 recall for the planted anomaly. This validates cursor state and selectors. It does not establish that a language model will choose the right filter, aggregate, or stopping point.

8.5 Transport policy

The adaptive policy keeps same-process state effectively upfront because no network transfer is needed. For each non-co-located topology, both 10K cases and both 100K cases are upfront under the configured threshold. At 1M, low expected reuse is on-demand and high expected reuse is upfront. This behavior matches the declared rule, but no latency optimum follows from it. A deployment must measure serialization, network bandwidth, backing-store latency, cache reuse, and the probability of expansion before selecting thresholds.

9 Security and Reliability Contract

Typed adaptive context creates capabilities to reveal hidden state. Its security boundary must therefore be explicit.

- Authorization precedes full reads, selected ranges, cursor opens, and every cursor page operation.
- Record IDs and cursor handles are opaque locators, never bearer authorization.
- Tenant and session scope are part of identity resolution; the test suite rejects cross-session reads.
- TTL and explicit deletion revoke state; content hashes detect local payload corruption before decode.
- Audit rows record ingestion, expansion, denial, cursor open, and cursor fetch events with scope and timestamp.
- Redaction is not yet implemented. A production policy must apply it to both compact and full views so expansion cannot bypass disclosure controls.

The store currently writes local plaintext. Encryption, key rotation, distributed authorization, concurrent mutation, cursor revocation propagation, and audit durability remain open production gates.

10 Relationship to Prior Systems

Prompt-compression methods such as LLMLingua and LongLLMLingua learn or compute shorter prompt representations under token budgets [1, 2]. Paper 7 is orthogonal to the choice of compressor: it specifies where an exact original lives and how different views are addressed and materialized. A learned compressor can be registered without becoming the backing state.

MemGPT frames long-context management as movement between memory tiers and uses explicit control flow to manage what enters context [3]. Paper 7 shares the tiered-state intuition but centers typed tool results, selective fields/ranges, deployment transport, and cursor operations. Retrieval-augmented generation supplies external evidence to a generator [4]; here, a RAG result is itself one typed record that may be compacted, re-addressed, and progressively inspected.

Headroom's public Compress–Cache–Retrieve documentation describes type-aware tool-output compression, local hash-indexed original caching, a model retrieval tool, markers, and proactive expansion [5]. That architecture is a strong reversible baseline. Paper 7 does not claim that Headroom fails. It asks a different evaluation question: when the compact view omits the fact that would make retrieval relevant, what address or policy keeps the downstream action reachable? The current mechanism check confirms explicit recovery and leaves latent recovery open.

Paper 6.5's typed capability work separates broad tool/skill discovery from bounded selection and exact full-schema activation. Paper 7 preserves those capability records and adds the result side of the interaction: what happens to a typed observation after a tool executes. The companion Paper 3.1 motivates multiple address views by showing that one generated summary can be a brittle retrieval key. The present study tests that design principle with deterministic records rather than repeating its summarization experiments.

11 Limitations and Next Gates

The study is an inception checkpoint. Its 45 records are synthetic and small, and the five seeds vary trigger placement rather than sampling a population of real agent tasks. No language model decides whether to materialize, chooses a cursor operation, or produces an answer. Consequently:

1. explicit address recall is an index capability result, not end-to-end evidence or answer recall;
2. proactive action probing is an oracle-like upper bound because the candidate action is supplied;
3. cursor correctness validates data operations, not autonomous analytics;
4. topology measurements are policy accounting, not physical network or distributed-store benchmarks;
5. active native-K/V is estimated from serialized bytes only when a model- specific bytes-per-token value is configured;
6. dense, learned Q/K, generated-summary, redaction, remote-store, and distributed cursor implementations remain absent.

The next experiment should put a frozen model in the loop on held-out explicit, latent, and action-trigger tasks. It should compare full context, compact-only, CCR-style marker/tool retrieval, multi-view indexed retrieval, proactive expansion, mixed cursors, and an oracle materialization policy at matched visible-token and call budgets. Primary outcomes should include task success, TriggerRecall, wrong-action rate, expansion precision, bytes, active K/V, round trips, TTFT, and wall time. Real database, log, graph, RAG, file, and API workloads should replace synthetic fixtures before default-policy claims.

12 Conclusion

Agent context is structured state with a lifecycle, not merely text waiting to be truncated. Paper 7 implements a first unified substrate: typed result records, deterministic compact views, independent retrieval addresses, exact scoped backing state, progressive materialization, transport policy, and stateful cursors. The mechanism study confirms exact recovery, scope checks, explicit addressability, and bounded cursor operations while reducing visible bytes.

It also rejects an easy conclusion. Lossless backing storage does not make a hidden action trigger visible, and an index cannot search for a discriminator that neither the query nor policy supplies. Reversible compression is therefore necessary infrastructure, not a complete adaptive-context policy. The next gate is behavioral: whether models and proactive policies can discover when, where, and how much to materialize without surrendering the savings that made compression useful.

A Public API Sketch

The facade keeps deployment policy separate from records:

```
policy = ContextPolicy(
    storage="adaptive",
    local_store="./tmp/pra",
    retrieval_mode="mixed",
    topology="remote_model",
    record_policies={
        RecordType.DB_RESULT: TypeContextPolicy(unit_limit=8),
    },
    cursor_policy=CursorPolicy(page_size=20, max_page_size=200),
)
runtime = AdaptiveContextRuntime(
    RecordScope(tenant_id="tenant-a", session_id="session-42"),
    policy,
)
record = runtime.ingest(
    tool_result, record_type=RecordType.DB_RESULT,
    provenance={"tool": "query_orders", "call_id": "call-17"},
)
```

The initial prompt receives the compact view. Discovery code may inspect the record's address views without exposing the original. Exact identity materialization is shared by native and tool interfaces:

```
selected = runtime.materialize(MaterializationEvent(
    record_id=record.record_id,
    level="selected",
    selector={"rows": [100, 120]},
))
full = runtime.retrieve_record(record.record_id, level="full")

cursor = runtime.open_cursor(
    record.record_id,
    collection="rows",
    filters={"region": "west"},
)
page = runtime.fetch_cursor(cursor.cursor_id)
stats = runtime.cursors.aggregate(
    cursor.cursor_id, "latency_ms", scope=runtime.scope
)
```

B Implementation Map

Module	Responsibility
`src/pra_hf/context_records.py`	shared record types and named view vocabulary inherited by capability and result records
`src/pra_hf/context_store.py`	scope, content identity, canonical encoding, manifests, TTL, hash verification, capacity, deletion
`src/pra_hf/typed_context.py`	compressor registry, compact/address views, adaptive record descriptor, deterministic selectors
`src/pra_hf/adaptive_context_runtime.py`	public policy, transport decision, materialization, search, cursor operations, accounting, audit
`experiments/paper7_records/run_inception_experiments.py`	five-seed fixtures, metrics, JSON artifacts, TeX macros, and figures

C Reproducibility

From the repository root:

```
python -m pytest tests/test_context_store.py \
  tests/test_typed_context.py \
  tests/test_adaptive_context_runtime.py -q
python experiments/paper7_records/run_inception_experiments.py
```

Raw per-record, per-mode, cursor, and transport rows are under `docs/papers/shared/results/paper7_records/inception`. The same command regenerates the

summary, TeX macros, and both figures. Unit tests additionally cover content-address stability, binary persistence, corruption detection, TTL expiration, cross-session denial, selected-range validation, cache reuse, cursor bounds, filters, search, aggregation, and closure.

References

[1] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of EMNLP*, 2023.

[2] Huiqiang Jiang, Qianhui Wu, Xufang Luo, et al. LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. In *Proceedings of ACL*, 2024.

[3] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023.

[4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, 2020.

[5] Headroom Labs. Reversible Compression (CCR): Compress–Cache–Retrieve architecture. `https://docs.headroomlabs.ai/docs/ccr`, accessed August 2026.